

Comment le texte arrive au modèle ?

Des caractères aux tokens, des tokens aux vecteurs

Une phrase apparaît à l'écran. Nous la lisons immédiatement. Nous voyons des mots, une syntaxe, peut-être une intention. Le modèle, lui, ne reçoit rien de tout cela sous cette forme. Il ne voit ni phrase, ni mot, ni idée. Il reçoit une suite d'éléments transformés en nombres.

Cette réponse paraît simple. Elle ne l'est pas. Dire que le texte devient des nombres ne suffit pas, car plusieurs opérations se succèdent et chacune répond à un problème différent. Il faut distinguer ce qui permet de conserver le texte, ce qui permet d'en désigner les unités et ce qui permet enfin de calculer sur elles. Autrement dit, le texte doit devenir stockable, puis indexable, puis calculable.

Voilà le passage que nous allons suivre. Non pas pour réduire le langage à une mécanique, mais pour comprendre précisément à quel moment la mécanique commence.

1. Le modèle ne reçoit pas des mots

Commençons par une difficulté presque invisible. Lorsque nous lisons le mot « manger », nous avons l'impression qu'il forme naturellement une unité. C'est un mot, donc ce sera sans doute une unité pour le modèle. Très bien. Mais que faire d'« anticonstitutionnellement » ? Que faire d'un nom propre rare, d'une faute de frappe, d'un terme scientifique ou d'un mot inventé hier soir dans une cour de récréation ?

Le système ne peut pas posséder une entrée séparée pour chaque forme possible. Il découpe donc le texte en unités appelées **tokens**. Un token peut correspondre à un mot entier, à une partie de mot, à un signe de ponctuation ou à une autre unité définie par le tokenizer. Les formes fréquentes ont davantage de chances d'être conservées en un seul morceau. Les formes rares sont souvent fragmentées.

« Manger » pourra ainsi correspondre à un seul token, tandis qu'« anticonstitutionnellement » pourra être découpé en plusieurs fragments. Le découpage exact dépend du modèle et de son vocabulaire. Les fragments que l'on donne dans une explication pédagogique sont donc des exemples, non une loi universelle.

On pourrait objecter que cette opération abîme déjà le langage. Le mot n'est-il pas une unité plus réelle qu'un fragment statistique ? Pour nous, souvent. Pour le modèle, la question est différente. Il ne cherche pas d'abord l'unité la plus fidèle à notre grammaire ; il cherche un compromis praticable entre la taille de son vocabulaire et sa capacité à représenter des textes très variés.

Le token n'est donc pas une définition du mot. C'est une unité de travail.

2. Un numéro qui ne signifie rien

Une fois le texte découpé, chaque token reçoit un identifiant numérique. Imaginons que le token « chat » porte le numéro 428. Il serait tentant de croire que ce nombre contient déjà

quelque chose du chat : sa place parmi les animaux, sa proximité avec le chien, peut-être même son goût regrettable pour les claviers d'ordinateur. Il n'en est rien.

Le numéro est arbitraire. Il ne décrit pas le token. Il indique seulement où aller chercher.

La meilleure image est celle du vestiaire. Le ticket 428 ne contient aucune propriété du manteau. Il ne dit ni sa couleur, ni sa taille, ni à qui il appartient. Il permet seulement de retrouver le bon casier. De la même manière, l'identifiant d'un token sert d'adresse.

Il faut ici distinguer la désignation et la représentation. L'identifiant désigne une unité. Il ne la représente pas encore d'une manière utile au calcul. Un nombre isolé comme 428 ne permet pas de dire que « chat » est plus proche de « chien » que de « carburateur ». Il donne une adresse, rien de plus.

Cette distinction est modeste, mais elle évite une confusion tenace : le modèle ne raisonne pas sur les numéros des tokens comme nous raisonnerions sur des quantités. Le token 428 n'est pas « plus grand » dans le langage que le token 127. L'ordre des identifiants n'a pas de sens sémantique.

3. Le casier contient un vecteur

Que trouve-t-on lorsque l'on ouvre le casier ? Une ligne de nombres.

Le modèle possède une matrice de plongement, souvent appelée *token embedding matrix*. Elle comporte une ligne pour chaque token du vocabulaire. Dans de nombreux modèles contemporains, cela représente des dizaines de milliers, parfois de l'ordre de cent mille tokens ou davantage, et chaque ligne contient un grand nombre de valeurs.

L'identifiant permet d'extraire la ligne correspondante. Le token 428 conduit, par exemple, à une suite de nombres comme : 0,21 ; -0,47 ; 0,83 ; 0,14 ; et ainsi de suite. Cette suite forme un **vecteur**.

Pourquoi compliquer ainsi une chose aussi simple qu'un mot ? Parce qu'un mot ne peut pas entrer directement dans un calcul. Un vecteur, oui. On peut comparer deux vecteurs, mesurer une distance ou un angle entre eux, les additionner, les transformer et les faire circuler dans les couches du réseau.

Le passage décisif se trouve là. Le token cesse d'être seulement une unité repérée dans une liste. Il reçoit des coordonnées dans un espace mathématique.

On dit parfois que le modèle « comprend » alors le mot. La formule va trop vite. Le vecteur rend le token calculable ; il ne prouve aucune expérience intérieure et ne constitue pas une définition de dictionnaire. Il faut résister à deux erreurs symétriques : croire que ces nombres ne sont qu'un code sans structure, ou croire qu'ils contiennent déjà le sens comme une petite essence enfermée dans une boîte.

Ils sont appris pour être utiles à la prédiction. C'est moins magique. C'est aussi plus intéressant.

4. D'où viennent les coordonnées ?

Au commencement, les nombres de la matrice ne sont pas déposés par un linguiste. Aucun dictionnaire universel ne vient inscrire que le chat est un animal, que Paris est une ville ou que le passé s'oppose au futur. Les valeurs sont initialisées, puis modifiées pendant l'apprentissage.

Le modèle s'entraîne, de manière simplifiée, à prédire le token suivant. Il reçoit un contexte, produit une prédiction, compare cette prédiction à la suite réelle, puis ajuste ses paramètres pour réduire l'erreur. L'opération est répétée un nombre immense de fois.

Prenons une phrase élémentaire : « Le chat dort sur... » Le système propose une suite. Sa prédiction est comparée au texte observé. L'écart produit une correction. Cette correction ne réécrit pas une règle dans un cahier secret ; elle déplace légèrement de nombreuses valeurs numériques.

De répétition en répétition, une géométrie se forme.

Pourquoi « chat » et « chien » finissent-ils souvent par apparaître dans des positions comparables ? Non parce qu'une autorité a déclaré : « Voici deux animaux domestiques. » Ils se rencontrent dans des environnements linguistiques partiellement semblables. On les nourrit, on les adopte, ils dorment, courent, mordent parfois le facteur ou ignorent superbement leur propriétaire. Les contextes ne sont pas identiques, mais ils se recouvrent suffisamment pour produire des régularités.

On pourrait objecter que le modèle ne voit alors que des voisinages de mots et jamais les choses elles-mêmes. C'est exact. Il apprend à partir des traces présentes dans son corpus. Mais cette limite n'annule pas le mécanisme ; elle en précise la nature. La matrice ne contient pas le monde. Elle contient une organisation numérique devenue utile pour prédire du texte sur le monde.

5. Une carte sans territoires

Pourquoi parler de carte ? Parce que chaque vecteur fournit des coordonnées. Si nous rassemblons les lignes de la matrice, chaque token occupe une position relative aux autres.

Mais cette carte n'est pas une carte géographique ordinaire. Elle ne possède ni nord, ni frontière administrative, ni région appelée « animaux » que l'on pourrait colorier en vert. Elle comporte souvent des milliers de dimensions, bien au-delà de ce que nous pouvons représenter sur une page.

Une illustration en deux dimensions n'en montrera donc qu'une projection simplifiée. Elle peut faire apparaître des voisinages ou des directions, mais elle écrase nécessairement une grande partie de la structure. L'image aide. Elle ment aussi un peu. Toute bonne carte le fait : elle choisit ce qu'elle rend visible.

Il faut surtout distinguer la carte et la géométrie. La carte désigne l'ensemble des positions. La géométrie désigne les relations que l'on peut mesurer entre elles : distances, angles, déplacements, directions approximatives.

La position absolue d'un token importe peu. Ce qui compte, ce sont les relations. « Chat » peut être proche de « chien » selon certaines dimensions, mais aussi se rapprocher de « tigre », de « moustaches », de « ronronner » ou d'une quantité d'autres termes selon les

régularités apprises. Le sens n'est pas rangé dans une case unique. Il est distribué dans un réseau de relations.

Voilà pourquoi il serait trompeur d'imaginer une colonne intitulée « animal », une autre « féminin », une autre « passé ». Les propriétés utiles ne sont généralement pas isolées dans un emplacement lisible. Elles apparaissent dans des configurations distribuées.

6. Les directions qui apparaissent

Les anciens modèles de plongements statiques ont rendu célèbre une opération : « roi » moins « homme » plus « femme » donne un point proche de « reine ». L'exemple a beaucoup circulé, parfois au point de devenir une petite légende de l'intelligence artificielle.

Que montre-t-il réellement ? Une régularité approximative. Le déplacement qui mène d'« homme » à « femme » peut ressembler à celui qui mène de « roi » à « reine », ou d'« oncle » à « tante ». Une relation apparaît alors comme une direction dans l'espace.

Très bien. Mais il ne faut pas conclure qu'une flèche unique contiendrait « le genre » de manière pure et définitive. Les plongements sont imparfaits, dépendants du corpus, du modèle et de la méthode utilisée. Les analogies ne fonctionnent pas toujours. Elles peuvent aussi reproduire des biais présents dans les textes d'entraînement.

L'exemple reste pourtant précieux. Il montre qu'une relation peut émerger sans avoir été explicitement écrite sous la forme d'une règle symbolique. Le système n'a pas reçu une table disant : homme correspond à femme, roi correspond à reine, oncle correspond à tante. Il a ajusté ses paramètres pour prédire, et certaines régularités se sont organisées géométriquement.

Le même principe peut faire apparaître des relations grammaticales : aller et allé, prendre et pris, singulier et pluriel, présent et passé. Là encore, aucune colonne ne porte nécessairement le nom de la règle. La régularité existe comme une structure distribuée.

Ce n'est pas une grammaire écrite dans un manuel. Ce n'est pas non plus un pur chaos numérique. C'est une organisation apprise.

7. Le premier vecteur n'est qu'un départ

Nous pourrions croire que le travail est terminé : le texte a été découpé, chaque token a trouvé son vecteur, la phrase peut entrer dans le modèle. En réalité, le passage n'est pas l'arrivée.

Le vecteur extrait de la matrice est initialement le même chaque fois que le même token apparaît. Prenons « avocat ». Dans « consulter un avocat » et « couper un avocat », le token commence avec le même plongement initial, alors que nous comprenons immédiatement qu'il ne désigne pas la même chose.

La matrice fournit donc un point de départ, pas le sens définitif dans la phrase.

Le transformeur confronte ensuite chaque représentation aux autres éléments du contexte. À travers ses couches, il modifie progressivement la représentation du token. « Avocat » ne

sera plus représenté de la même manière lorsqu'il est entouré de « tribunal », « plaidoirie » et « client » que lorsqu'il est entouré de « salade », « noyau » et « citron ».

Il faut alors distinguer le plongement initial du token et sa représentation contextuelle. Le premier est extrait de la matrice et reste stable pour ce token. La seconde est produite par le traitement de la phrase et change selon l'occurrence.

C'est là que commence véritablement le travail du contexte. Les tokens ne restent pas posés sur une carte comme des villes immobiles. Leurs représentations sont déplacées par la phrase en cours.

8. Ce que ce passage nous oblige à comprendre

Nous étions partis d'une phrase visible à l'écran. Nous découvrons maintenant une succession de transformations : des caractères encodés, des tokens, des identifiants, des vecteurs initiaux, puis des représentations contextuelles.

On pourrait résumer tout cela en disant : « Le texte devient des nombres. » La phrase est correcte. Elle est aussi insuffisante. Elle efface les distinctions qui permettent de comprendre le mécanisme.

Les caractères rendent le texte enregistrable. Les tokens le découpent en unités manipulables. Les identifiants permettent de retrouver ces unités. La matrice leur donne des coordonnées. L'apprentissage organise progressivement ces coordonnées. Le transformeur les déplace ensuite selon le contexte.

À chaque étape, quelque chose est gagné et quelque chose est perdu. Le découpage rend le texte calculable, mais il ne respecte pas toujours nos unités linguistiques spontanées. La géométrie fait apparaître des relations, mais elle ne livre pas des définitions transparentes. Le contexte produit des représentations fines, mais celles-ci deviennent plus difficiles à interpréter directement.

Voilà le point décisif : le modèle ne reçoit pas le sens comme un objet déjà formé. Il construit des représentations numériques parce qu'elles l'aident à accomplir sa tâche de prédiction. Nous pouvons ensuite y observer des régularités qui ressemblent à des catégories, des relations et des directions.

La machine ne commence donc pas par comprendre une phrase comme nous la comprenons. Elle commence par la transformer en un espace où des calculs deviennent possibles. Le reste de son travail se déroule dans cet espace.

La question suivante ne sera plus : comment le texte entre-t-il dans le modèle ? Elle sera plus difficile : que devient-il une fois qu'il y est entré ?